

文件系统 API

1.shell命令

ls pwd cd mkdir rm cat echo cp mv df insmod lsmod rmmmod

2.文件读写API

文件读写相关 API 在 dfs_posix.h 中声明

```
int open(const char *file, int flags, ...)
```

功能：打开文件并根据指定的打开标志返回文件描述符

参数：

```
const char *file    //文件的路径名
int flag            //文件的打开标志，可组合多个标志
```

返回值：

```
成功：>=0
失败：负数
```

flags 介绍：

O_RDONLY: 以只读方式打开文件

O_WRONLY: 以只写方式打开文件

O_RDWR: 以读写方式打开文件

O_CREAT: 如果改文件不存在，就创建一个新的文件，并用第三个参数为其设置权限

O_EXCL: 如果使用O_CREAT时文件存在，则返回错误消息。这一参数可测试文件是否存在。此时open是原子操作，防止多个进程同时创建同一个文件

O_NOCTTY: 使用本参数时，若文件为终端，那么该终端不会成为调用open()的那个进程的控制终端

O_TRUNC: 若文件已经存在，那么会删除文件中的全部原有数据，并且设置文件大小为0

O_APPEND: 以添加方式打开文件，在打开文件的同时，文件指针指向文件的末尾，即将写入的数据添加到文件的末尾

O_NONBLOCK: 如果pathname指的是一个FIFO、一个块特殊文件或一个字符特殊文件，则此选择项为此文件的本次打开操作和后续的I/O操作设置非阻塞方式。

O_SYNC: 使每次write都等到物理I/O操作完成。

"注意：在open()函数中，falgs参数可以通过"|"组合构成，但前3个标准常量（O_RDONLY，O_WRONLY，和O_RDWR）不能互相组合。"

```
int close(int d)
```

功能：关闭打开的文件描述符

参数：

```
int d                //文件描述符
```

返回值：

```
成功：0
失败：-1
```

```
int read(int fd, void *buf, size_t len)
```

功能：指定数据缓冲区长度并对打开的文件描述符进行读取

参数：

```
int fd           //文件描述符
void *buf        //缓冲区用于保存读取的数据
size_t len       //数据缓冲区的最大长度
```

返回值：

成功：返回实际的读取数据缓冲区长度，如果返回值为0，则可能到达文件末尾，请检查errno

失败：-1

"注意：errno 的值需使用 fs_set_errno()函数获取"

```
int write(int fd, const void *buf, size_t len)
```

功能：往打开的文件描述符写入指定的数据缓冲区长度

参数：

```
int fd           //文件描述符
const void *buf   //要写入的数据缓冲区
size_t len       //数据缓冲区的长度
```

返回值：

成功：返回实际写入数据缓冲区的长度

失败：-1

```
off_t lseek(int fd, off_t offset, int whence)
```

功能：用于在指定的文件描述符中将文件指针定位到相应位置

参数：

```
int fd           //文件描述符
off_t offset      //要设置的偏移量
int whence        //偏移的起始位置
```

返回值：

成功：文件中当前位置

失败：-1

whence 介绍

SEEK_SET: 当前位置为文件的开头，新位置为偏移量的大小

SEEK_CUR: 当前位置为指针的位置，新位置为当前位置加上偏移量

SEEK_END: 当前位置为文件的结尾，新位置为文件大小加上偏移量的大小

```
int rename(const char *old, const char *new)
```

功能：将文件重命名

参数：

```
const char *old   //旧文件名
const char *new    //新文件名
```

返回值：

成功：0

失败：-1

```
int unlink(const char *pathname)
```

功能：从文件系统取消链接（删除）指定的路径文件

参数：

```
const char *pathname //要取消链接（删除）的指定路径名
```

返回值：

成功：0

失败：-1

```
int stat(const char *file, struct stat *buf)
```

功能：获取文件信息

参数：

```
const char *file      //文件的文件名
struct stat *buf      //用来保存统计信息的数据缓冲区
```

返回值：

```
成功：0
失败：-1
```

```
int fstat(int fildes, struct stat *buf)
```

功能：获取文件状态

参数：

```
int fildes           //文件描述符
struct stat *buf     //用来保存同统计信息的数据缓冲区
```

返回值：

```
成功：0
失败：-1
```

```
int fsync(int fildes)
```

功能：将fildes命名的打开文件描述符的所有数据都传输到与fildes描述的文件关联的存储设备

参数：

```
int fildes           //文件描述符
```

返回值：

```
成功：0
失败：-1
```

```
int fcntl(int fildes, int cmd, ...)
```

功能：设置和修改描述符的属性

参数：

```
int fildes           //文件描述符
int cmd              //指定的命令
...                  //特定设备执行请求的功能所需的其他信息
```

返回值：

```
成功：0
失败：-1
```

```
int ioctl(int fildes, int cmd, ...)
```

功能：在设备上执行各种控制功能，提供对连接到fd的设备驱动程序的特性和操作的访问

参数：

```
int fildes           //文件描述符
int cmd              //指定的命令
...                  //特定设备执行请求的功能所需的其他信息
```

返回值：

```
成功：0
失败：返回 -1 并返回 errno设置为指示错误
```

```
int mkdir(const char *path, mode_t mode)
```

功能：创建一个目录

参数：

```
const char *path    //要创建的目录路径
mode_t mode         //参数模式
```

返回值：

```
成功：0
失败：非0
```

```
DIR *opendir(const char *name)
```

功能：打开一个目录

参数：

```
const char *name    //要打开的路径名
```

返回值：

```
成功：返回目录的DIR指针
失败：NULL
```

```
struct dirent *readdir(DIR *d)
```

功能：返回该目录流中的下一个目录条目

参数：

```
DIR *d              //目录流指针
```

返回值：

```
成功：下一个目录条目
失败：NULL
```

```
long telldir(DIR *d)
```

功能：返回目录流中的当前位置

参数：

```
DIR *d              //目录流指针
```

返回值：

```
成功：目录流中的当前位置
失败：0
```

```
void seekdir(DIR *d, off_t offset)
```

功能：设置目录流中下一个目录结构的位置

参数：

```
DIR *d              //目录流指针
off_t offset        //目录流中的偏移量
```

返回值：无

```
void rewinddir(DIR *d)
```

功能：重置目录流

参数：

```
DIR *d              //目录流指针
```

返回值：无

```
int closedir(DIR *d)
```

功能：关闭目录流

参数：

```
DIR *d          //目录流指针
```

返回值：

成功：0

失败：-1

```
int rmdir(const char *pathname)
```

功能：删除目录

参数：

```
const char *pathname    //要删除的路径名
```

返回值：

成功：0

失败：非0

```
int chdir(const char *path)
```

功能：更改工作目录

参数：

```
const char *pathname    //要更改的路径名
```

返回值：

成功：0

失败：-1

```
char *getcwd(char *buf, size_t size)
```

功能：返回当前工作目录的绝对路径并将其复制到buf中

参数：

```
char *buf          //返回当前工作目录
```

```
size_t size        //缓冲区大小
```

返回值：当前工作目录

```
int statfs(const char *path, struct statfs *buf)
```

功能：返回有关已安装文件系统的信息

参数：

```
const char *path    //挂载文件系统的路径
```

```
struct statfs *buf   //缓冲区用于保存返回的信息
```

返回值：

成功：0

失败：非0

```
int access(const char *path, int amode)
```

功能：根据amode中包含的位模式检查path参数所指向的路径名所命名的文件的可访问性

参数：

```
const char *path    //指定的文件/目录路径
```

```
int amode            //amode值是要检查的访问权限(R_OK, W_OK, X_OK)或存在性测试(F_OK)的按位或
```

返回值：

成功：0

失败：-1